



Penetration Testing

Arcane 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Arcane team partnered with Hashlock to conduct a security Penetration Test of their Arcane Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

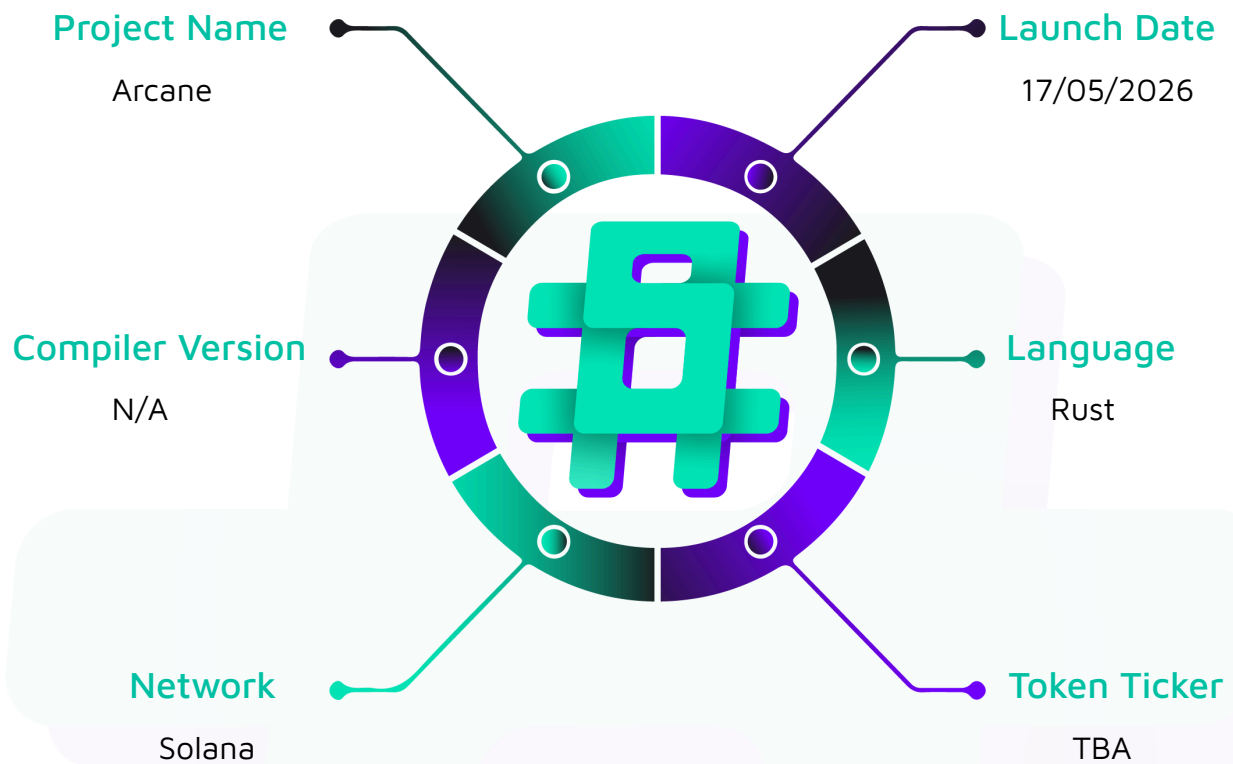
Project Name: Arcane

Project Type: DeFi

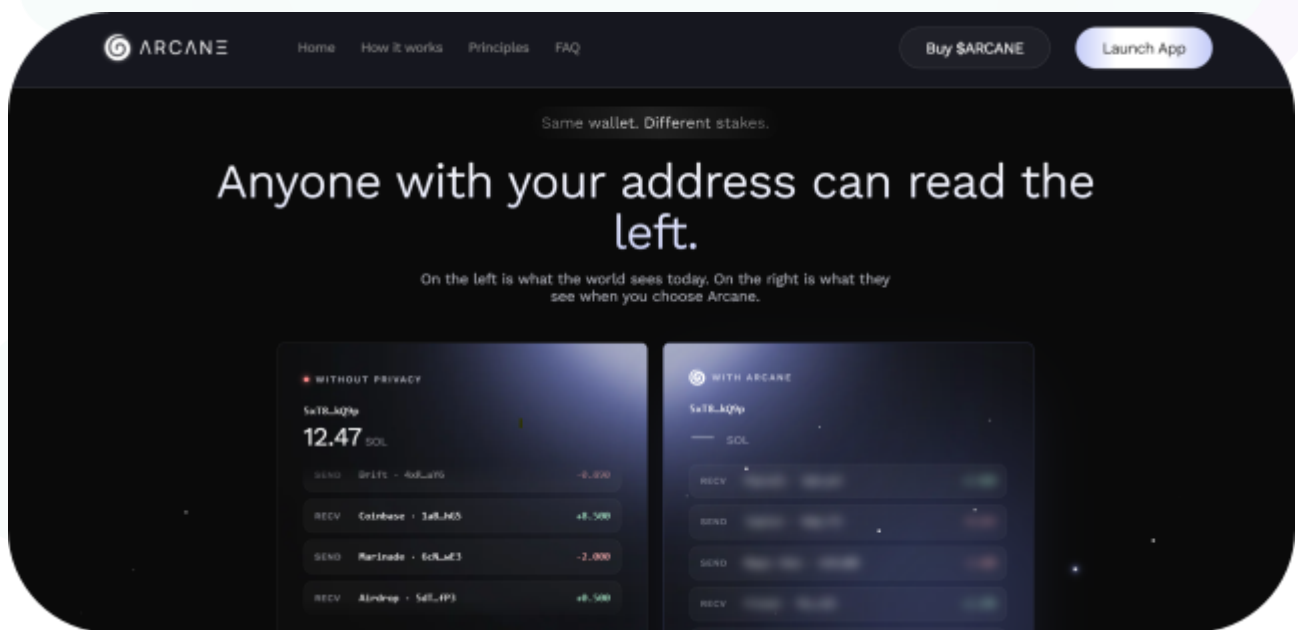
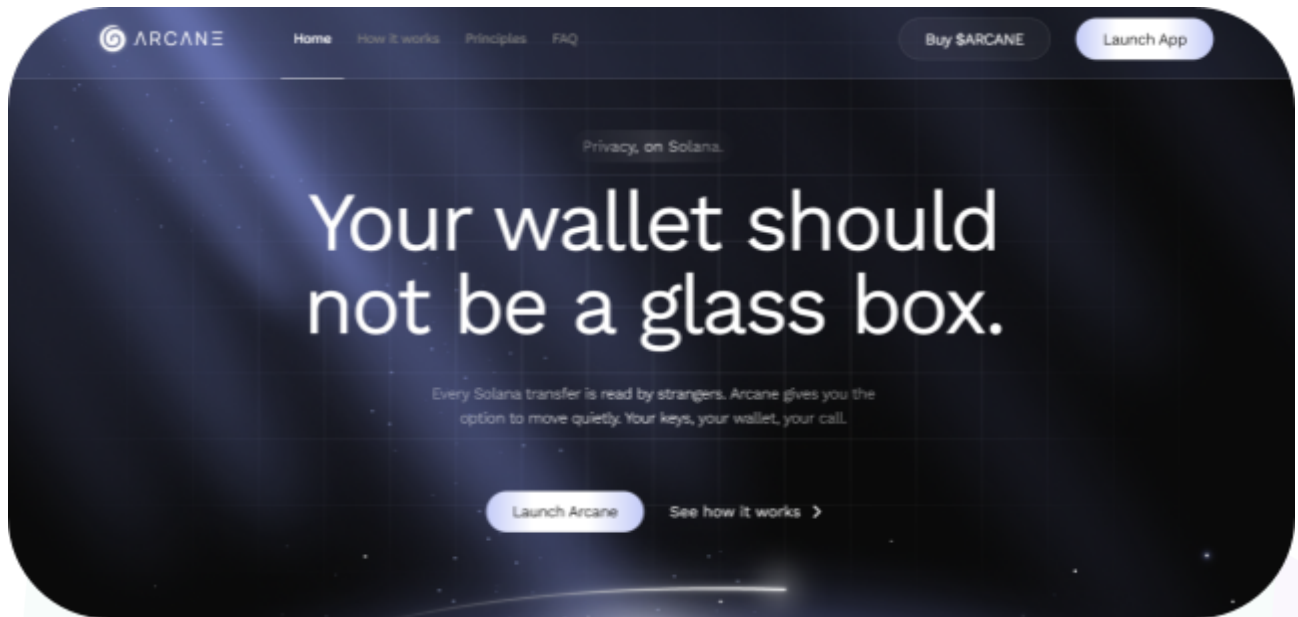
Website: <https://arcaneprivacy.com/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Arcane project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Arcane Project code base
Platform	Solana
Penetration Test Date	July, 2026
Repository/Website URL/IP Tested	https://github.com/ArcanePrivacy/sdk
GitHub Commit Hash (If applicable)	a4ca9eb6380045e4c0104cc4ffd6ee62afd4a119
Fix Review GitHub Commit Hash (If applicable)	e40e65e7b9fc77d3f4f0caeb49a2e335894fb85b

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01][arc-sdk] circuits/constants#resolveNpmCdnCircuitBase - circuit wasm and proving key are fetched from a public CDN with no integrity pinning

Description

In production browsers the witness-generation `withdraw.wasm` and the proving `withdraw.zkey` default to a version-pinned `jsDelivr` URL with no subresource-integrity hash, no signature, and no local fallback.

Vulnerability Details

These artefacts are handed to `snarkjs.groth16.fullProve`, which processes the private `nullifier` and `secret` inputs during witness generation. The delivery path is therefore trusted with spend-equivalent secret material.

```
// src/circuits/constants.ts:28-30
export function resolveNpmCdnCircuitBase(): string {
  return
  `https://cdn.jsdelivr.net/npm/${PACKAGE_NAME}@${PACKAGE_VERSION}/dist/circuits/`;
}
```

Impact

A compromised CDN entry, a maliciously republished package version, or a CDN man-in-the-middle can serve a backdoored `withdraw.wasm` that exfiltrates the user's spend-equivalent `secret` and `nullifier` during witness generation, or silently miscomputes the proof.

Recommendation

Pin the circuit artefacts by content hash, verifying the hash after fetch and failing closed on mismatch, and prefer bundling or self-hosting them under the application origin.

Status

Resolved

Low

[L-01][arc-sdk] arcaneProgram#getEncryptionKey - note-backup encryption key derived from a signature over a fixed constant with no domain separation

Description

The `getEncryptionKey` function has the wallet sign the static string `arcane:notes:v1` and takes `nacl.hash(signature).slice(0, 32)` as the symmetric key that encrypts the user's secret notes for on-chain backup. The signed message is not bound to the program, network, origin, wallet, or any nonce.

Vulnerability Details

Because `ed25519` signatures are deterministic, the same key is reproducible by anyone who can induce the wallet to sign this exact constant. The derived key is the sole protection over the secret notes backed up on-chain through `backupNotes`, and the secret note is the only spending authority for a deposit.

```
// src/arcaneProgram.impl.js:612-627
export async function getEncryptionKey(wallet) {
  const message = new TextEncoder().encode('arcane:notes:v1');
  let signature;
  if (wallet instanceof KeyPair) {
    signature = nacl.sign.detached(message, wallet.secretKey);
  } else if (typeof wallet.signMessage === 'function') {
    signature = await wallet.signMessage(message);
  }
  const key = nacl.hash(signature);
  return key.slice(0, 32);
}
```

Impact

Any dApp or phishing site that convinces the victim to sign the innocuous-looking string `arcane:notes:v1` derives the identical note-encryption key, decrypts the victim's on-chain backup notes, recovers the secret notes, and can withdraw the funds those notes control.

Recommendation

Derive the key from a domain-separated, context-rich message that includes the program ID, network, chain, wallet public key, and a versioned scheme tag, ideally through a labelled HKDF or SLIP-0021 construction. Never derive a spend-equivalent key from a globally reusable constant.

Status

Acknowledged

[L-02] [arc-sdk] rangeSDK - range third-party API key is shipped in the client bundle and forwarded to oracle operators

Description

The documented frontend wiring sources the Range API key from `import.meta.env.VITE_RANGE_API_KEY`, and Vite inlines any `VITE_` value into the shipped client bundle. The key is a real third-party credential for the Range risk API.

Vulnerability Details

The key is also passed as `variableOverrides.RANGE_API_KEY` into a Switchboard on-demand oracle job through the shared public `CrossbarClient.default()`, where it becomes the `X-API-KEY` header value sent to `api.range.org`.

```
// src/rangeSDK.ts:19,62-66
headers: [ { key: "X-API-KEY", value: "${RANGE_API_KEY}" } ],
const sigVerifyIx = await queue.fetchQuoteIx(crossbarClient, [feed], {
  variableOverrides: { RANGE_API_KEY: rangeApiKey },
});
```

Impact

Any visitor can read the Range API key out of the client bundle and abuse it for quota exhaustion, billing, or impersonated risk queries, and the oracle and Crossbar operators observe the key in transit.

Recommendation

Route Range risk lookups through a backend proxy that holds the key server-side and never ship it in a client bundle. If the oracle-job design requires the key on the client, treat it as public and scope and rotate it aggressively.

Status

Acknowledged

[L-03][arc-sdk] config/constants#resolveNetworkFromRpcEndpoint - network is inferred from an RPC URL substring and silently defaults to mainnet

Description

The `resolveNetworkFromRpcEndpoint` function returns `devnet` only when the lower-cased RPC endpoint contains `devnet` or `dev`, and otherwise returns `mainnet`. The `dev` substring is broad enough to match unrelated hostnames, and a private devnet RPC lacking either token silently resolves to mainnet.

Vulnerability Details

A misclassified network selects the wrong indexer URL, token mints, deposit queue, and address lookup table. Because the mainnet lookup table is empty, a devnet user misclassified as mainnet hits a hard failure, while a mainnet user misclassified as devnet would transact against devnet mints.

Impact

The SDK interacts with the wrong network's mints and indexer while connected to the other cluster, causing failed operations or transactions issued under an incorrect network assumption.

Recommendation

Require an explicit network option or match against exact known-cluster hostnames, throw when the cluster is ambiguous rather than defaulting silently, and at minimum remove the overly broad `dev` alternative.

Status

Resolved



Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.